

80. Regular Expressions: Character Classes

Ek practical kaam: phone number `" +7(903)-123-45-67"` ko sirf numbers `79031234567` me badalna. Iske liye jo number nahi hai use dhundh ke hata do. **Character classes** yahi madad karti hain.

Character class ek special notation hai jo kisi certain set ke **koi bhi ek symbol** ko match karta hai.

`\d` (digit)

"Koi bhi ek digit" (`0` se `9`).

```
js
let str = "+7(903)-123-45-67";

/\d/      // pehla digit: 7
/\d/g     // saare digits: 7,9,0,3,1,2,3,4,5,6,7

str.match(/\d/g).join('');    // 79031234567
```

`g` ke bina sirf pehla match.

Sabse zyaada use hone wali classes

- `\d` ("d" = digit): `0` se `9`
- `\s` ("s" = space): space, tab `\t`, newline `\n` aur kuch rare (`\v`, `\f`, `\r`)
- `\w` ("w" = word): Latin letter, digit ya underscore `_`. **Non-Latin letters (cyrillic, hindi) `\w` me nahi aate.**

`\d\s\w` = "digit, phir space symbol, phir wordly character", jaise `1 a`.

Regexp me normal symbols aur character classes dono ho sakte hain:

```
js
"Is there CSS4?".match(/CSS\d/);           // CSS4
"I love HTML5!".match(/\s\w\w\w\w\d/);     // ' HTML5'
```

Inverse classes

Har class ka "ulta" hota hai, wahi letter **uppercase** me. Ye baaki sab characters match karta hai:

- `\D`: non-digit (digit ke alawa kuch bhi, jaise letter)

- `\S`: non-space (space ke alawa kuch bhi)
- `\W`: non-wordly (`\w` ke alawa sab, jaise non-Latin letter ya space)

Phone number me se digits nikalne ka chhota tarika: non-digits hata do:

```
js
"+7(903)-123-45-67".replace(/\D/g, "");    // 79031234567
```

Dot `.`: "koi bhi character"

Dot ek special class hai: **newline ke alawa koi bhi character.**

```
js
"Z".match(/./);           // Z

let regexp = /CS.4/;
"CSS4".match(regexp);     // CSS4
"CS-4".match(regexp);     // CS-4
"CS 4".match(regexp);     // CS 4 (space bhi character hai)
```

Dot ka matlab "koi bhi character", **na ki "character ka na hona"**. Character hona zaruri hai:

```
js
"CS4".match(/CS.4/);      // null, kyunki dot ke liye koi character nahi
```

`s` flag se dot literally sab kuch

By default dot newline `\n` ko match nahi karta:

```
js
"A\nB".match(/A.B/);      // null
"A\nB".match(/A.B/s);     // A\nB (match!)
```

Kai situations me dot ko literally "koi bhi character, newline bhi" chahiye, uske liye `s` flag.

`s` flag **IE me supported nahi**. Har jagah chalne wala alternative: `[\s\S]` ("space ya non-space", yaani kuch bhi). Ye "Sets and ranges [...]" chapter me aayega. Koi bhi complementary pair (jaise `[\d\D]`) chalta hai, ya `[^]` bhi.

```
js
```

```
"A\nB".match(/A[\s\S]B/); // A\nB (match!)
```

Ye trick tab bhi kaam aati hai jab ek hi pattern me dono chahiye: normal dot (newline chhodke) aur "kuch bhi" (`[\s\S]`).

Spaces par dhyan do

Aam taur par hum spaces par dhyan nahi dete: `1-5` aur `1 - 5` lagbhag same lagte hain. Par agar regexp spaces ko count nahi karta to fail ho sakta hai:

```
js
"1 - 5".match(/\d-\d/); // null, match nahi
"1 - 5".match(/\d - \d/); // 1 - 5, chalega
"1 - 5".match(/\d\s-\s\d/); // 1 - 5, ye bhi chalega
```

Space ek character hai, kisi bhi dusre character jitna hi important. Regexp me spaces jodne ya hatane par wo waisa hi kaam nahi karega. Regexp me har character matter karta hai, spaces bhi.

Summary

- `\d`: digits
- `\D`: non-digits
- `\s`: space symbols, tabs, newlines
- `\S`: `\s` ke alawa sab
- `\w`: Latin letters, digits, underscore `_`
- `\W`: `\w` ke alawa sab
- `.`: `s` flag ke saath koi bhi character, warna newline `\n` ke alawa koi bhi

Par ye sab nahi hai! JS ke strings Unicode use karte hain, jisme characters ki kai properties hoti hain (letter kis language ka hai, punctuation hai ya nahi, etc.). In properties se bhi search kar sakte hain, uske liye `u` flag chahiye (agla chapter).